技術選定 Pros / Cons

1. 目的

本資料では、現在の実装で使用している技術について、採用理由とメリット・デメリットを整理する。

今回のシステムでは、オンライン診療中の**不審発言検知を主機能**とし、音声・映像の特徴を補助情報として、なりすましの疑いを評価する。

未決定の技術については、無理に採用を決めず、今後の検証対象として扱う。

2. 技術選定の基本方針

- 現在の実装資産を可能な限り活用する。
- 不審発言検知を中心に設計する。
- なりすましの疑いは補助的な評価として扱う。
- 機能ごとの責務を分離し、将来の技術変更に対応できる構成にする。
- 精度・応答速度・費用・プライバシーを考慮する。
- 未検証の機能を採用済みとして扱わない。

3. 現在の実装・構成で使用している技術のPros / Cons

3.1 Chrome Extension / Manifest V3

Pros（メリット）	Cons（デメリット）
Chrome上で動作し、Google Meetのタブ音声を取得できる。	Chromeに依存するため、他のブラウザやオンライン会議サービスへの対応が必要。
`tabCapture` とOffscreen Documentを組み合わせられる。	参加者ごとの音声トラックを直接取得できない。
マイク入力とタブ音声を組み合わせられる。	マイクとタブ音声の混合・分離や音質の検証が必要。
既存のPoCを活用できる。	現在は録音停止後にBlobを生成して保存する方式であり、リアルタイム分析には追加実装が必要。

評価：継続利用の候補

既存PoCを活用できるため、音声取得の基盤として継続する方針とする。ただし、音声取得の品質と安定性は引き続き検証する。

3.2 Next.js / React

Pros（メリット）	Cons（デメリット）
Reactを利用したコンポーネント設計ができる。	リアルタイム更新の状態管理を設計する必要がある。

Pros（メリット）	Cons（デメリット）
画面と機能を分割して管理できる。	サーバー側処理とブラウザ側処理の責務を明確にする必要がある。
既存のフロントエンド実装を活用できる。	リアルタイム通信や長時間接続には追加設計が必要。

評価：継続利用

既存画面とAPIクライアントを活用できるため、現時点で変更する必要性は低い。

3.3 TypeScript

Pros（メリット）	Cons（デメリット）
型によってデータ構造やAPIの誤りを検出しやすい。	Python側の型・スキーマと整合性を保つ必要がある。
フロントエンドの保守性を高められる。	型定義を変更した場合、関連する箇所も更新する必要がある。
エディタの補完や静的解析を利用できる。	実行時の入力値検証を型だけで代替することはできない。

評価：継続利用

フロントエンドの型安全性を維持し、バックエンドとのAPI契約を明確にする。

3.4 Zustand

Pros（メリット）	Cons（デメリット）
状態管理を簡潔に記述できる。	状態が増えると責務が複雑になる可能性がある。
Reactコンポーネント間で状態を共有できる。	更新処理や非同期処理の管理方法を統一する必要がある。
既存実装を活用できる。	大規模化した場合はStoreの分割が必要になる。

評価：継続利用

現在のフロントエンドの状態管理に使用しているため、現時点では継続する。

3.5 FastAPI / Python

Pros（メリット）	Cons（デメリット）
PythonのAI関連ライブラリと連携しやすい。	非同期処理やストリーミング処理の設計が必要。
APIの定義と入力値検証を行いやすい。	フロントエンドのTypeScriptとのスキーマ整合性を管理する必要がある。

Pros（メリット）	Cons（デメリット）
既存のAPI・Detector構造を活用できる。	処理量が増えた場合、メモリ使用量や処理時間の検証が必要。

評価：継続利用

既存のFastAPI実装とPythonのAI関連資産を活用する。現時点でTypeScriptへのバックエンド移行は行わない。

3.6 Gemini / Google Gen AI SDK

Pros（メリット）	Cons（デメリット）
音声・映像や付随するコンテキストを入力として、分析結果や理由を構造化して返す実装が可能。	判定が確率的で、誤検知や見逃しが発生する可能性がある。
自然言語の文脈を理解し、分析理由を文章で提示できる。	判定が確率的で、誤検知や見逃しが発生する可能性がある。
既存のGeminiDetectorを活用できる。	API利用料金や応答時間を考慮する必要がある。
構造化出力を利用できる。	出力形式の検証や異常応答への対応が必要。

評価：既存のメディア分析実装を活用

現在のDetectorのデフォルト設定はMockDetectorである。GeminiDetectorにはGemini APIとVertex AIを利用するコードが存在するが、実環境での有効化・利用状況は未確認である。そのため、Vertex AIを採用済みとは扱わない。

3.7 Cloud Run

Pros（メリット）	Cons（デメリット）
コンテナ単位でアプリケーションをデプロイできる。	コールドスタートによる応答遅延が発生する場合がある。
HTTPSやオートスケーリングに対応する。	同時接続や長時間接続の設計が必要。
FastAPIなど既存のコンテナ構成を利用できる。	インスタンス間でセッション状態を共有する場合は追加設計が必要。

評価：今後の採用候補

既存のGCP環境を活用できるため、バックエンドのデプロイ先として採用を検討する。実際のデプロイや稼働は未実施であり、リアルタイム通信を導入する場合は、接続時間・再接続・同時接続数などを検証する。

4. 現時点の選定方針

分類	技術	方針
音声取得	Chrome Extension / tabCapture	既存PoCを継続して検証
フロントエンド	Next.js / React	継続利用
言語	TypeScript	継続利用
状態管理	Zustand	継続利用
バックエンド	FastAPI / Python	継続利用
AI連携	Gemini / Google Gen AI SDK	既存実装を活用し、用途ごとに検証
インフラ	Cloud Run	採用候補（デプロイ・稼働は未実施）

5. 未決定事項

以下は現時点で採用を確定していないため、本資料では技術選定の結論を出さない。

- 音声認識サービスの選定
- 不審発言検知におけるAIモデルとルールベースの役割分担
- なりすまし疑い評価における音声・映像分析の具体的な方式
- リアルタイム通信方式
- Firestoreなどの永続化の要否
- 音声・文字起こし・検知結果の保存範囲と保存期間

これらは、精度・応答速度・費用・プライバシー要件を確認したうえで別途決定する。

6. セキュリティ・プライバシー上の留意事項

- 音声・映像・文字起こしには、個人情報や診療に関する機微情報が含まれる可能性がある。
- 取得・分析・外部送信の目的を利用者に説明し、適切な同意を得る必要がある。
- 必要以上の音声・映像・文字起こしを保存しない。
- ログに会話内容や認証情報を含めない。
- 外部AIサービスの利用条件、データ処理、保持期間、リージョンを確認する。
- 検知結果は確定的な判断ではなく、利用者の確認を支援する情報として扱う。

7. 今後の更新方針

技術選定を変更した場合は、以下を更新する。

- 本資料の技術選定表
- `docs/architecture.md` の目標アーキテクチャ
- API仕様・スキーマ
- 関連するADR（Architecture Decision Record）

採用が確定していない技術を、採用済みとして記載しない。